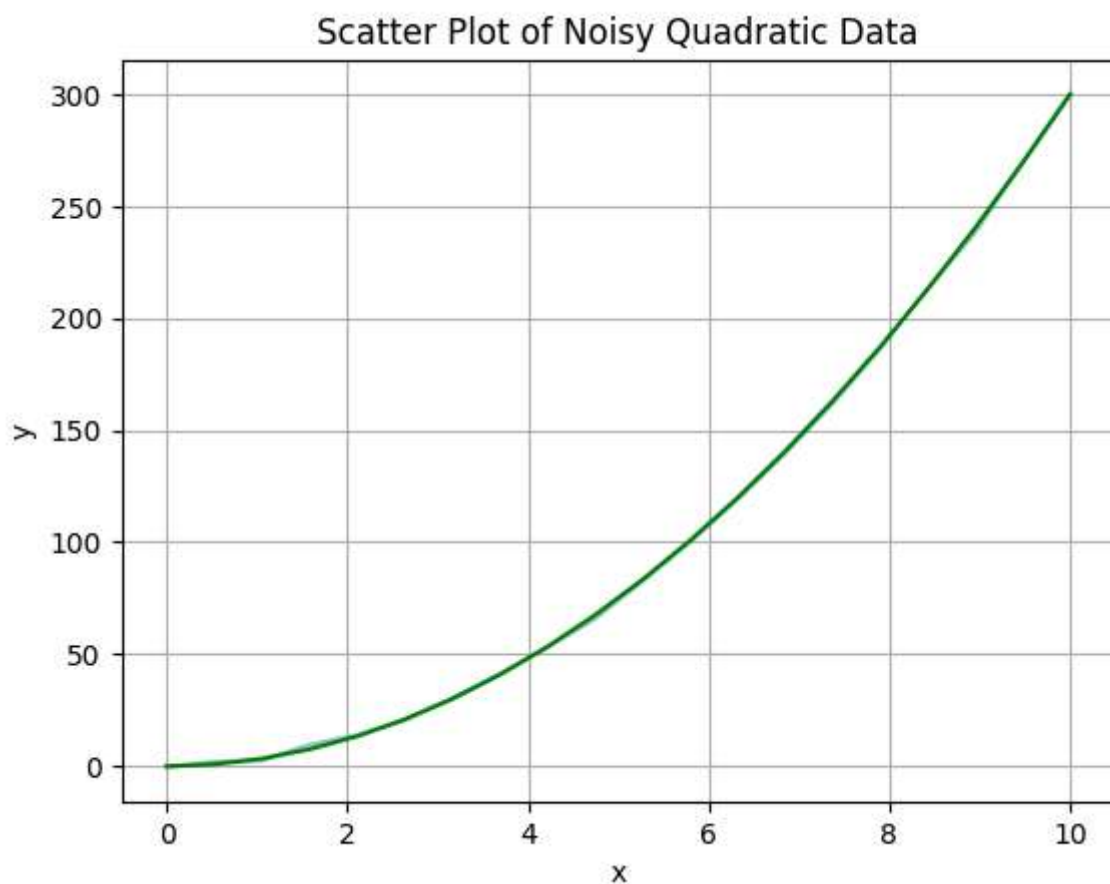


Fit $y = 3x^2 + 5 + N(0,1)$ with polynomial functions

```
In [6]: import numpy as np
import matplotlib.pyplot as plt
```

Original curve with noise

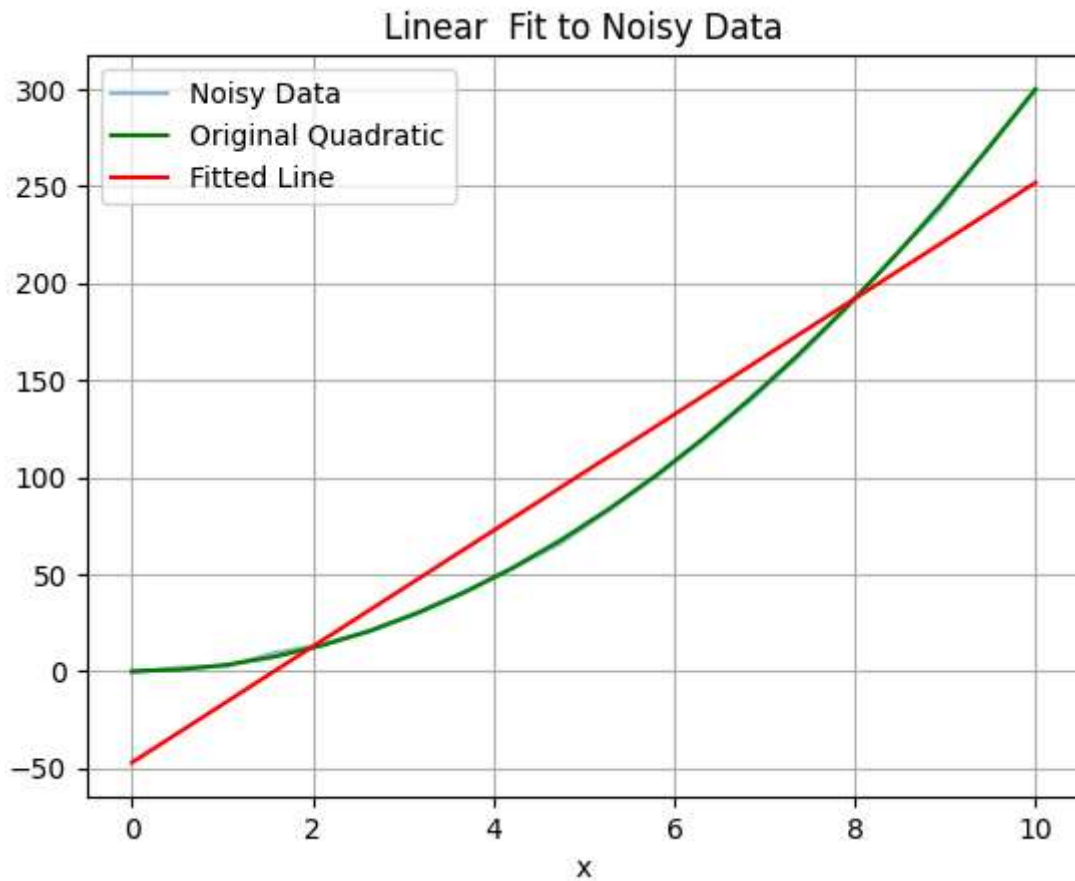
```
In [7]: x = np.linspace(0, 10, 20)
y = 3 * x**2 + np.random.normal(0, 1, size=x.shape)
y Og = 3 * x**2
plt.plot(x, y, alpha=0.5)
plt.plot(x, y Og, color='green', label='Original Quadratic')
plt.title('Scatter Plot of Noisy Quadratic Data')
plt.xlabel('x')
plt.ylabel('y')
plt.grid()
plt.show()
```



Fittnig 1-D line

```
In [8]: X = np.column_stack((np.ones_like(x), x))
theta = np.linalg.inv(X.T @ X) @ X.T @ y
y_pred = X @ theta
print("Coefficients:", theta)
plt.title('Linear Fit to Noisy Data')
plt.xlabel('x')
plt.plot(x, y, alpha=0.5, label='Noisy Data')
plt.plot(x, y Og, color='green', label='Original Quadratic')
plt.plot(x, y_pred, color='red', label='Fitted Line')
plt.legend()
plt.grid()
plt.show()
```

Coefficients: [-47.05702537 29.88043086]

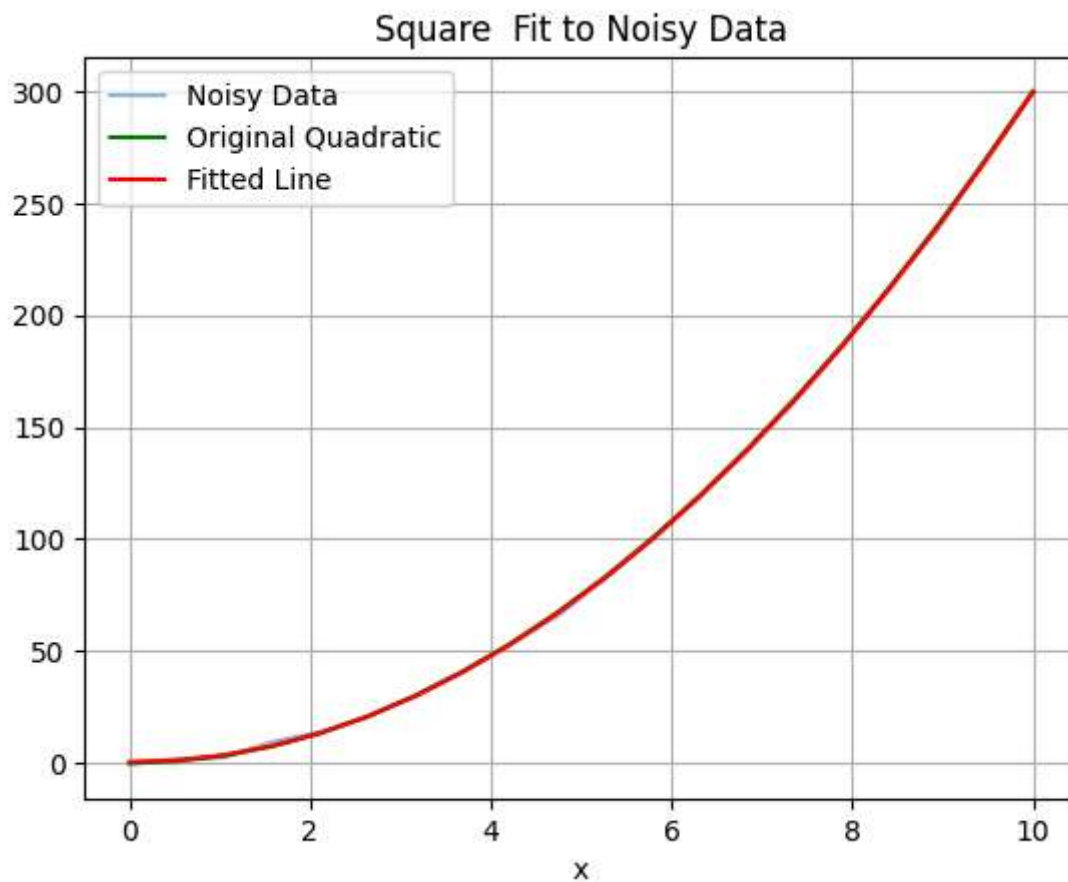


Fitting square curve

```
In [9]: X = np.column_stack((np.ones_like(x), x, x**2))
theta = np.linalg.inv(X.T @ X) @ X.T @ y
y_pred = X @ theta
print("Coefficients:", theta)
plt.title('Square Fit to Noisy Data')
plt.xlabel('x')
plt.plot(x, y, alpha=0.5, label='Noisy Data')
plt.plot(x, y Og, color='green', label='Original Quadratic')
plt.plot(x, y_pred, color='red', label='Fitted Line')
plt.legend()
```

```
plt.grid()
plt.show()
```

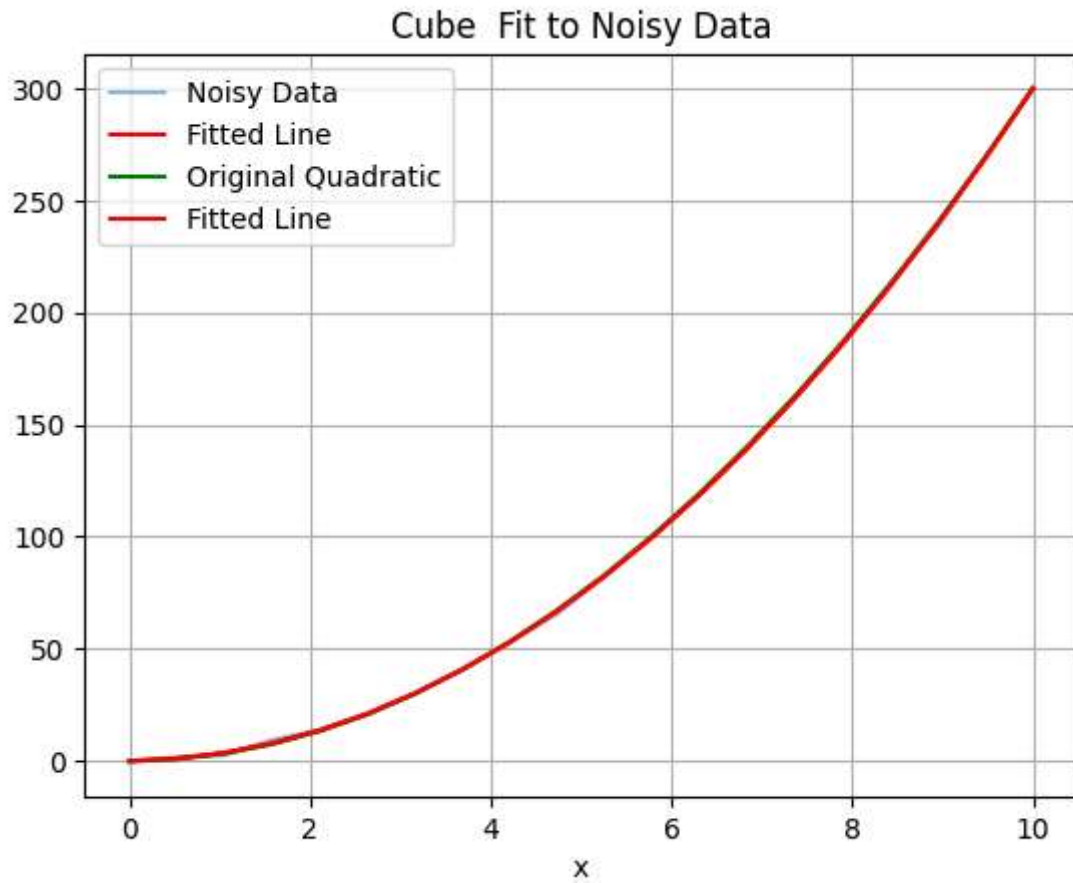
Coefficients: [0.57827096 -0.28859014 3.0169021]



Fitting cubinc curve

```
In [10]: X = np.column_stack([np.ones_like(x), x, x**2, x**3])
theta = np.linalg.inv(X.T @ X) @ X.T @ y
y_pred = X @ theta
plt.plot(x, y, alpha=0.5, label='Noisy Data')
plt.plot(x, y_pred, color='red', label='Fitted Line')
print("Coefficients:", theta)
plt.title('Cube Fit to Noisy Data')
plt.xlabel('x')
plt.plot(x, y Og, color='green', label='Original Quadratic')
plt.plot(x, y_pred, color='red', label='Fitted Line')
plt.legend()
plt.grid()
plt.show()
```

Coefficients: [0.00383486 0.5039815 2.81359677 0.01355369]



In []: